Les Mathématiques d'Alzheimer

Alzheimer Maths

Yami

1 Les fondations du système (axiomes)

1.1 Le set de départ

L'univers est initialisé avec un inventaire fini contenant les entiers $[1, \ldots, 9]$, leurs négatifs $[-1, \ldots, -9]$ et leurs fractions inverses $[1, \ldots, 1/9]$.

Le 1 ici est considéré comme une itération différente de 1, mais agit comme 1.

Et leurs fractions inverses négatives.

1.2 Axiome 1 : consommation destructive

Toute opération mathématique consomme définitivement ses opérandes. Un nombre utilisé est détruit et n'existe plus, sauf s'il est recréé par le résultat d'une opération.

Les opérations qui existent sont la multiplication et l'addition, et agissent comme dans le système ZFC.

1.3 Axiome 2 : multiplicité

Un même nombre peut exister en plusieurs itérations (jetons) distinctes dans l'inventaire.

1.4 Axiome 3 : opération P (perte)

L'opération $P(x)$ consomme le jeton x sans générer de nouveau résultat. C'est une destruction pure.

1.5 Axiome 4 : opération R (reset)

L'opération R purge l'inventaire courant et réinitialise le système exactement à son état de départ.

1.6 Axiome 5

Un même terme, s'il n'a qu'une seule itération, ne peut pas être utilisé deux fois dans la même opération.

Par exemple, si l'ensemble contient seulement $[1, 0]$, l'opération

$$1 - 1$$

est interdite.

2 Les théorèmes fondamentaux

2.1 Théorème 1 : le nombre de nombres

« Vous aurez toujours moins de nombres dans le système qu'au départ. »

Démonstration. Chaque opération binaire ($+$, $\times$, $-$, $\div$) consomme deux jetons pour n'en générer qu'un seul (bilan : -1 jeton).

Chaque opération P consomme un jeton pour en générer zéro (bilan : -1 jeton).

La quantité globale de jetons décroît strictement à chaque étape.

2.2 Théorème 2 : la fin du monde

« Si vous commencez avec un ensemble de N nombres, vous aurez au maximum N opérations dans ce système, et la N-ième opération différente de R sera toujours un P. »

Démonstration. D'après le théorème 1, chaque opération retire exactement un jeton du système.

Après $N-1$ opérations, l'inventaire ne contient plus qu'un seul jeton :

$$N - (N-1) = 1.$$

À cet état, les opérations binaires sont impossibles par manque d'opérandes. La seule action légale restante, différente de R, est l'opération unitaire P.

La N-ième opération qui n'est pas un R est donc fatalement une extinction par P.

2.3 Théorème 3 : nombre maximal possible

Sachant que, pour deux entiers positifs a et b différents de 1, on a :

$$a * b \geq a + b,$$

le nombre maximal possible dans notre set est obtenu en multipliant les entiers positifs et les facteurs négatifs appropriés.

On peut utiliser au maximum huit facteurs négatifs :

$$(-9)(-8)(-7)(-6)(-5)(-4)(-3)(-2) = 9!.$$

On a donc :

$$(9!)^2.$$

Et on peut ajouter à ce nombre la somme des décimales, de 1 à $\frac{1}{9}$:

$$(9!)^2 \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9}\right)^2.$$

Appelons ce nombre, par simplicité, Max.

Tout autre moyen est non optimal. Additionner les termes négatifs réduit la valeur, multiplier les fractions réduit la valeur et additionner des valeurs positives crée des nombres moins grands que leur multiplication.

Il s'agit de l'ensemble des stratégies possibles, en prenant les ensembles terme par terme.

Pour les nombres de 1 à 9, on peut les additionner ou les multiplier. Or, puisque

$$a * b \geq a + b,$$

les multiplier crée alors un nombre plus grand.

Pour les négatifs, on peut les additionner ou les multiplier. Les additionner crée des nombres plus négatifs. Même si on essaie de les additionner puis de les multiplier par -1, on créera un nombre moins grand qu'une grande multiplication.

Pour créer un nombre encore plus grand avec le premier 9!, l'optimal est de les multiplier à nouveau.

Pour les fractions positives, sachant qu'elles sont comprises entre -1 et 1, les multiplier créera toujours des nombres plus petits. Les additionner est le choix optimal.

Leur somme sera supérieure à 1 ; multiplier est alors le choix optimal.

Pour les fractions négatives, la stratégie utilisant la logique des fractions est une somme utilisée comme terme d'un produit. Pour transformer la somme négative en terme positif d'un produit, on utilise le -1 laissé par les termes négatifs.

L'ensemble des termes créés est supérieur à 1. La manière optimale de les associer est alors la multiplication.

On obtient alors :

$$\boxed{\text{Max} = (9!)^2 \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9}\right)^2}$$

Comme

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9} = \frac{7129}{2520},$$

on a :

$$\boxed{\text{Max} = (362880)^2 \left(\frac{7129}{2520}\right)^2}$$

Numériquement :

$$\boxed{\text{Max} \approx 1.0538 \times 10^{12}}$$

3 Simulation Python

3.1 Vérification de la multiplicité des jetons

```python
def T2valide(t1, t2, nombres):
    if t1 != t2:
        return True
    else:
        n = 0
        for i in nombres:
            if i == t1:
                n += 1

        if n > 1:
            return True

    return False
```

3.2 Programme de simulation

```
print("On peut arreter la simulation en tapant stop")

Run = True
New_world = True


def T2valide(t1, t2, nombres):
    if t1 != t2:
        return True

    n = 0

    for i in nombres:
        if i == t1:
            n += 1

    return n > 1


while Run:

    if New_world:
        nombres = [
            # Entiers positifs
            1.0, 2.0, 3.0, 4.0, 5.0,
            6.0, 7.0, 8.0, 9.0,

            # Entiers negatifs
            -1.0, -2.0, -3.0, -4.0, -5.0,
            -6.0, -7.0, -8.0, -9.0,

            # Fractions positives
            1.0, 1/2, 1/3, 1/4, 1/5,
            1/6, 1/7, 1/8, 1/9,

            # Fractions negatives
            -1.0, -1/2, -1/3, -1/4, -1/5,
            -1/6, -1/7, -1/8, -1/9
        ]

        New_world = False
        print("Nouvel inventaire :", nombres)

    else:
        operation = input(
            "L'operation a effectuer (+, *, P, R, stop) : "
        )

        if operation == "stop":
```

```
            Run = False

        elif operation == "R":
            New_world = True

        elif operation == "P":
            T1 = float(input("Nombre a detruire : "))

            if T1 in nombres:
                nombres.remove(T1)
                print("Inventaire :", nombres)
            else:
                print("Operation impossible : jeton absent.")

        elif operation in ("+", "*"):
            T1 = float(input("Premier nombre : "))

            if T1 not in nombres:
                print("Premier jeton absent.")
                continue

            T2 = float(input("Deuxieme nombre : "))

            if T2 not in nombres or not T2valide(
                T1, T2, nombres
            ):
                print("Operation impossible : jetons indisponibles.
                    ")
                continue

            # Consommation des deux jetons
            nombres.remove(T1)
            nombres.remove(T2)

            # Creation du resultat
            if operation == "+":
                resultat = T1 + T2
            else:
                resultat = T1 * T2

            nombres.append(resultat)

            print("Resultat :", resultat)
            print("Inventaire :", nombres)

        else:
            print("Operation non acceptee.")
```